

Exact, Bounded, or Relaxed: Proving Which Floating-Point Guarantee a Compiler Rewrite Keeps

A Aswanth Raj

School of Computer Science and Engineering, VIT Vellore
Vellore, India

Dr. RANJITHKUMAR S

School of Computer Science and Engineering, VIT Vellore
Vellore, India

Abstract

Compilers offer a binary choice for floating-point code: strict IEEE 754 semantics, or “fast-math” permissions under which reassociation and identity rewrites may change results. The choice conflates three guarantees. A rewrite may be *bit-identical*, because every value it touches is exact or rounded once; it may change bits while keeping the source program’s worst-case error bound; or it may hold only over the real numbers. We show that a compiler can prove, per rewrite, which of the three it keeps, from declared input domains, and can certify every output with the weakest guarantee that reaches it. A grid-magnitude abstraction proves that a matrix chain whose proper sub-chains fit in 53 significand bits, and whose final sums round at most once, returns the correctly rounded product under every bracketing and every order of summation; one step above this threshold, inputs exist on which bracketings differ. The standard componentwise error bound of a chain is the same for every bracketing, which licenses bound-preserving reassociation under a range condition alone. In a matrix-language compiler that emits C, the proved reorderings make GCC-compiled kernels up to 278× faster with identical bits, and up to 469× faster within the source order’s error bound on real data, where `-ffast-math` gains at most 1.13×. Across 57,600 executions no output certified bit-identical differed from the unoptimized program; this check found, and we fixed, one unsound certificate in an earlier version.

Keywords: floating-point arithmetic, compiler optimization, fast-math, reassociation, abstract interpretation, numerical contracts, matrix chains

1 Introduction

Floating-point addition and multiplication are not associative, and $x \cdot 0 = 0$, $x + 0 = x$ and $x \cdot I = x$ fail for signed zeros, NaN and infinities [1, 2]. Compilers therefore forbid such rewrites by default and offer permissions that enable them: GCC’s `-ffast-math` and its components [3], or LLVM’s per-instruction fast-math flags [4]. A permission states what the compiler *may* do. It does not state what the result *is*: after `-fassociative-math` a programmer cannot tell which outputs still equal the strict program, which changed within a known error bound, and which changed arbitrarily.

These are three different guarantees, and a single switch conflates them. Consider the chain $R = ABC$ with $A, C \in$

Table 1. One chain $R = ABC$ (40×2 , 2×40 , 40×2) over three input domains: modeled arithmetic after optimization (11,120 in source order) and the certificate of `print(R)`, as our compiler reports them.

Inputs	strict	bounded	algebraic
int8	556, bit-identical	556, bit-identical	556, bit-identical
real(1)	11,120, bit-identical	556, bound-pres.	556, bound-pres.
real	11,120, bit-identical	11,120, bit-identical	556, relaxed

$\mathbb{R}^{100 \times 2}$ and $B \in \mathbb{R}^{2 \times 100}$. Source order costs 69,800 scalar operations; $A(BC)$ costs 1,396. If A , B and C hold 8-bit integers, every product and partial sum of either bracketing is an integer of at most 29 bits, so both bracketings compute the same bits: the rewrite is *bit-identical*, and a strict compiler may apply it. If the matrices hold real values in $[-1, 1]$, the bits change, yet the textbook componentwise error bound of the chain is the same for every bracketing: the rewrite is *bound-preserving*. If values are unbounded, one bracketing may overflow where the other does not, and the rewrite is *relaxed*: valid over the reals and nowhere else.

The difference between these cases is not in the program text but in the values the program will see (Table 1). We make that knowledge explicit: program inputs declare a value domain (`input(int8)`, `input(real(1))`), the runtime checks each input against its domain before execution, and the compiler derives from the domains facts precise enough to decide, rewrite by rewrite, which guarantee holds. Each output is then certified with the weakest guarantee among the rewrites that reach it, and the same facts tell a C compiler which sums it may reorder.

Contributions.

- A lattice of three numerical contracts (strict, bounded, algebraic) whose proof obligations are per rewrite, and per-output certificates that compose them, including the inheritance rule that keeps certificates sound when a proof reads facts produced by a weaker rewrite (Section 3).
- A grid-magnitude abstract domain and a theorem for matrix chains: if every proper sub-chain fits in 53 significand bits and the final sums round at most once, every bracketing and every order of summation returns the correctly rounded product, including the sign of zero (Section 4). The threshold it gives is

the true boundary on three of the four families we measured and one step from it on the fourth.

- A proof that the standard componentwise bound of a chain is invariant under bracketing, which reduces bound-preserving reassociation to a range check (Section 5), and a segmented chain-ordering algorithm that applies the cheapest bracketing each contract admits (Section 6).
- Certificates as code: a C backend that marks exactly the sums the facts let a production compiler reorder, as standard OpenMP reduction clauses, and that doubles as a second implementation of the machine's arithmetic (Section 8).
- An evaluation of recovery, certificate soundness, threshold tightness, end-to-end value under GCC, test adequacy by mutation, and a preregistered accuracy study (Section 10).

2 Setting

MatrixLang is a statically shaped, straight-line language over binary64 matrices and scalars. Its compiler (flex, bison, C11) performs shape checking, lowers to three-address code (TAC), optimizes, and executes on a typed stack virtual machine (VM). A matrix product entry is evaluated as $c_{ij} = (\dots((+0 + a_{i1}b_{1j}) + a_{i2}b_{2j})\dots) + a_{in}b_{nj}$; the accumulator starts at +0, each term is one rounded multiplication and one rounded addition in round-to-nearest, and no fused multiply-add is used (`-ffp-contract=off`). Outputs are compared in hexadecimal, which distinguishes every binary64 value including -0 and NaN.

The optimizer performs constant folding, common subexpression elimination, copy propagation and dead-code elimination, which are bit-exact; algebraic identity and zero rewrites ($x \cdot 1$, AI , $x + 0$, $A \cdot 0$, $0 \cdot A$); and matrix-chain reordering by dynamic programming over the modeled cost $mp(2n-1)$ of an $m \times n$ by $n \times p$ product. Before this work a single flag decided all value-changing rewrites at once.

Input domains. A declaration `matrix A[40,2] = input(int8);` introduces a run-time input whose values the compiler never sees. Domains are `bool`, `uint8`, `int8`, `uint16`, `int16`, `int32`, `int(lo,hi)`, `real` (any finite value), `real(B)` ($|x| \leq B$) and `real(lo,hi)`. Values arrive from files or a seeded generator and are checked before execution: finite, integral for integer domains, within bounds, and with -0 stored as $+0$ in integer and non-negative domains. The check is what makes domain-derived facts sound; without input declarations every value would be a compile-time constant and constant folding would subsume optimization.

3 Contracts and certificates

Definition 1 (Guarantee levels). *A rewrite of program P into P' is bit-identical if every output of P' equals the corresponding output of P bit for bit, for every input in the declared domains;*

bound-preserving if every output of P' satisfies the standard-model worst-case error bound of P 's evaluation order and NaN and infinity occur in P' exactly where they occur in P ; and relaxed otherwise, when it is an identity over the reals.

The levels are ordered bit-identical $<$ bound-preserving $<$ relaxed. A *contract* names the weakest level a rewrite may have and still be applied: *strict* (the default) admits only bit-identical rewrites, *bounded* admits bound-preserving ones, and *algebraic* admits all. Each candidate rewrite computes its level from the facts of its operands; the contract applies or declines it, and a declined rewrite is reported with the contract that would permit it. Constant folding, common subexpressions, copy propagation, dead-code elimination and double transposition are bit-exact and need no proof.

Certificates. Every TAC instruction carries the weakest level of the rewrites applied to it. The certificate of an output is the weakest level in the dependency cone of its print, together with the reasons recorded by those rewrites. The compiler prints, for example:

```
Contract: bounded. Every output must keep the
source order's worst-case error bound.
1. bit-identical    print(R)
   - A * B * C is exact: every intermediate of
     every bracketing needs at most 51 of 53
     significand bits
2. bound-preserving print(S)
   - X * B * C keeps the source-order error
     bound: |error| <= 2.89e-15 |A1|...|Ak|
     under every bracketing, and no
     intermediate can overflow
```

Inheritance. A dependency cone in the *optimized* program is not enough. In $R = 0 \cdot (1 \cdot (0 \cdot N)^T)$ with $N = -A$, the inner $0 \cdot N$ may be -0 ; the bounded contract rewrites it to $+0$ as a bound-preserving rewrite. The outer product is then provably $+0$, a bit-identical rewrite, but only because of the inner one, and once the outer rewrite drops every use of the inner product, dead-code elimination removes it and its label. Our soundness experiment (Section 10.2) found exactly this case in an earlier version. A rewrite now inherits the weakest level in the dependency cone of the operands its proof read, at the time of the rewrite.

4 Exactness from input domains

The grid-magnitude domain. A fact about a matrix X is a tuple (g, m, fin, nn, nz) : every entry is a multiple of 2^g ($g \geq -1074$), has magnitude at most m , is finite, is non-negative, and is not -0 , respectively. Integer domains give $g = 0$ and $m = \max(|lo|, |hi|)$; bounded reals give $g = -1074$ (every binary64 value is a multiple of 2^{-1074}) and $m = B$; unbounded reals give $m = \text{DBL_MAX}$. Literals give exact facts. Transfer functions use upward-rounded bound arithmetic: a sum takes the smaller grid and adds magnitudes; a product

of entries adds grids; a matrix product of inner dimension n has grid $g_a + g_b$ and magnitude $n m_a m_b$, inflated by $(1 + 2nu)$ when not provably exact. Non-finite or overflowing results lose every fact. A matrix-product entry is never -0 , because its accumulator starts at $+0$.

Lemma 4.1 (Rounding preserves grids). *If x is a real multiple of 2^g with $g \geq -1074$ and $|x| \leq \text{DBL_MAX}$, then $\text{fl}(x)$ is a multiple of 2^g , and $\text{fl}(x) = x$ when $|x| \leq 2^{g+53}$.*

Proof. Let $e = \lfloor \log_2 |x| \rfloor$; the spacing of binary64 near x is $2^{\max(e-52, -1074)}$. If $|x| < 2^{g+53}$ then $e - 52 \leq g$, so the spacing divides 2^g and x is representable. If $|x| = 2^{g+53}$, x is a power of two in range. Otherwise $e \geq g + 53$, the spacing 2^{e-52} is a multiple of 2^g , and so is every representable neighbor of x . $\square$

Lemma 4.2 (Sums from $+0$). *In round-to-nearest, a sum of two multiples of 2^{-1074} is -0 only if both addends are -0 . Hence a summation tree with a $+0$ leaf never returns -0 .*

Proof. An exact zero sum of addends that are not both -0 is $+0$, and a non-zero multiple of 2^{-1074} never rounds to zero. By induction, a subtree is -0 only if every leaf is. $\square$

Let $A_1 \cdots A_k$ be a chain with $A_l \in \mathbb{R}^{d_{l-1} \times d_l}$ and facts (g_l, m_l) . For $i \leq j$ let $G_{ij} = \sum_{l=i}^j g_l$ and $M_{ij} = \prod_{l=i}^j m_l \cdot \prod_{l=i}^{j-1} d_l$, which bounds every entry of $|A_i| \cdots |A_j|$. Sub-chain $i..j$ fits if $M_{ij} = 0$, or $G_{ij} \geq -1074$ and $M_{ij} \leq 2^{G_{ij}+53}$.

Theorem 4.3 (Correctly rounded chains). *Let every operand be finite and every proper sub-chain $i..j \neq 1..k$ fit. Let the whole chain fit, or satisfy $G_{1k} \geq -1074$, $M_{1k} \leq \text{DBL_MAX}$ and $(d_s - 1) M_{1s} M_{s+1,k} \leq 2^{G_{1k}+53}$ for every split $1 \leq s < k$. Then every bracketing, with the terms of every product entry summed in any order from a $+0$ accumulator, returns $\text{fl}(A_1 \cdots A_k)$ entrywise, and no entry is -0 . All such evaluations are bit-identical.*

Proof. Every non-root node T of a bracketing computes its proper sub-chain $P_{ij} = A_i \cdots A_j$ exactly. By induction on $T = LR$ splitting after s : the children are exact, every term $\ell_{ab} r_{bc}$ is a multiple of $2^{G_{is}+G_{s+1,j}} = 2^{G_{ij}}$, and the sum of any subset of the d_s terms has magnitude at most $d_s M_{is} M_{s+1,j} = M_{ij} \leq 2^{G_{ij}+53}$, so by Lemma 4.1 every product and every addition, in any order, is exact. If $M_{ij} = 0$, every term is an exact zero of finite factors.

The root, split after s , has exact children. If the whole chain fits, the root is exact as well. Otherwise every term is at most $M_{1s} M_{s+1,k}$ and every sum of at most $d_s - 1$ terms at most $(d_s - 1) M_{1s} M_{s+1,k} \leq 2^{G_{1k}+53}$, so all of them are exact (when $d_s = 1$, the single product is correctly rounded by IEEE 754). In any summation tree, the lowest node whose subtree holds all d_s terms adds two exact partial sums and returns $\text{fl}(c)$ for the exact entry c ; any addition above it adds only the accumulator's $+0$. Lemma 4.2 excludes -0 . Every bracketing and every order thus returns $\text{fl}(A_1 \cdots A_k)$. $\square$

A proper sub-chain must be exact because some bracketing uses it as an operand, and a rounded operand changes everything computed from it. Checking the whole chain alone is unsound: if the last operand is a zero matrix, $M_{1k} = 0$, yet $A_1 A_2$ may overflow to ∞ and $\infty \cdot 0 = \text{NaN}$ in source order, while the reordered chain computes 0 ; our mutation study contains this witness (Section 10.5). For $\text{int}(\emptyset, m)$ chains of shape $100 \times 2 \cdot 2 \times 100 \cdot 100 \times 2$ the binding condition is the source order's last product, and the largest admissible bound is $m^* = 35,697$, since $198 \cdot 35697^3 \leq 2^{53} < 198 \cdot 35698^3$. Requiring the whole chain to be exact stops at 35,578; Section 10.3 shows that 35,697 is the true boundary.

Corollary 4.4 (Free reductions). *A product LR with inner dimension d , finite operands, $g_L + g_R \geq -1074$ and $(d - 1) m_L m_R \leq 2^{g_L+g_R+53}$ gives the same bits for every order of summation of every entry.*

5 Bound-preserving reassociation

In the standard model $\text{fl}(a \circ b) = (a \circ b)(1 + \delta)$, $|\delta| \leq u = 2^{-53}$, absent underflow and overflow, an inner product of length n evaluated in any order satisfies $|\text{fl}(x^T y) - x^T y| \leq \gamma_n |x|^T |y|$ with $\gamma_n = nu / (1 - nu)$ [6].

Theorem 5.1 (Bracketing invariance). *For every bracketing T of $A_1 \cdots A_k$ evaluated without underflow or overflow,*

$$|\hat{T} - A_1 \cdots A_k| \leq \left(\prod_{l=1}^{k-1} (1 + \gamma_{d_l}) - 1 \right) |A_1| \cdots |A_k|$$

componentwise, where $d_1, \dots, d_{k-1}$ are the inner dimensions.

Proof. Write $\Pi_T = \prod (1 + \gamma_d)$ over the dimensions contracted inside T and $|A|_T$ for the product of the operands' absolute values. We show $|\hat{T} - P_T| \leq (\Pi_T - 1) |A|_T$ by induction. For $T = LR$ contracting d , let $\hat{L} = P_L + E_L$ and $\hat{R} = P_R + E_R$ with $|E_L| \leq (\Pi_L - 1) |A|_L$, so $|\hat{L}| \leq \Pi_L |A|_L$. Then $\hat{T} = \hat{L} \hat{R} + F$ with $|F| \leq \gamma_d |\hat{L}| |\hat{R}|$ and $\hat{L} \hat{R} - P_L P_R = E_L P_R + P_L E_R + E_L E_R$. Using $|P_L| \leq |A|_L$,

$$\begin{aligned} |\hat{T} - P_T| &\leq [(\Pi_L - 1) + (\Pi_R - 1) + (\Pi_L - 1)(\Pi_R - 1) \\ &\quad + \gamma_d \Pi_L \Pi_R] |A|_T = (\Pi_L \Pi_R (1 + \gamma_d) - 1) |A|_T. \end{aligned}$$

Every inner dimension is contracted at exactly one node of any bracketing, so $\Pi_T = \prod_l (1 + \gamma_{d_l})$ for every T . $\square$

The bound coefficient depends neither on the bracketing nor, since γ_d holds for every order, on the order of summation inside each product, so a rewrite between bracketings preserves it whenever both are evaluated without overflow, and NaN and infinity cannot arise in either. The bounded contract therefore admits a reordering when every sub-chain's magnitude bound stays below 2^{1020} (leaving slack for the Π factors). This is why $\text{real}(1)$ chains are reordered under the bounded contract while chains over unbounded real are not: there no fact excludes overflow. Theorem 5.1 also

Table 2. Levels of identity and zero rewrites from operand facts. “fin” is finite, “nz” has no -0 entry, “nn” is non-negative.

Rewrite	bit-identical if	else
$x \cdot 1 \rightarrow x$ (scalar unit)	always	–
$AI, IA \rightarrow A$ (product)	fin, nz	bounded if fin
$x + 0, 0 + x \rightarrow x$	x nz	bounded
$x - 0 \rightarrow x$	x nz or zero nz	bounded
$A \cdot 0 \rightarrow 0$ (product)	other operand fin	relaxed
$0 \cdot A \rightarrow 0$ (scaling)	both nn and fin	bounded if fin

explains why an earlier attempt to trade cost against worst-case accuracy in the chain dynamic program was degenerate: all bracketings tie. Underflow adds an absolute term to the standard model; we do not claim that term is bracketing-invariant (Section 12).

6 Segmented chain ordering

A contract may admit a cheaper bracketing for part of a chain but not the whole. For a chain $A_1 \cdots A_k$ and source-order prefixes $P_i = A_1 \cdots A_i$ (computed left to right, as the source does), the pass computes

$$best[j] = \min_{i < j} best[i] + \text{cost}(P_i \cdot A_{i+1} \cdots A_j),$$

where a segment of two operands is the source step, always legal, and a longer segment may use its cheapest bracketing (the classical $O(k^3)$ program [7]) if its level is admitted: bit-identical by Theorem 4.3 for strict, bit-identical or range-safe for bounded, anything for algebraic. Segment facts use the facts of P_i as their first operand. Ties prefer the stronger guarantee, so an exact tree is chosen over a bound-preserving tree of equal cost. Only segment roots carry a guarantee: interior products have no counterpart in the source program, and labelling them let common subexpression elimination spread one chain’s proof to an unrelated output. On a four-operand $\text{int}(\emptyset, m)$ chain $24 \times 3 \cdot 3 \times 24 \cdot 24 \times 3 \cdot 3 \times 24$, the whole chain is bit-identical only for $m \leq 2,568$, but its first three operands are up to $m = 50,727$, and the strict contract reorders that segment (Section 10.3). Operands built in place, such as $\text{ones}(n, 1)$ in A^31 , read nothing and are moved above the chain so that its products are contiguous.

7 IEEE side conditions of identity rewrites

Table 2 lists the conditions. A matrix product with I is not the identity in IEEE 754: each entry is $+0 + a \cdot 1 + b \cdot 0 + \cdots$, which maps -0 to $+0$ and ∞ to NaN through $\infty \cdot 0$. Adding $+0$ maps -0 to $+0$. A scaling takes the sign of each factor, so $0 \cdot (-3) = -0$. The rule $0 - x \rightarrow -x$ is absent: it maps $+0 - (+0) = +0$ to -0 and saves no arithmetic.

8 Certificates as code

A guarantee is worth most when a production compiler can act on it. `matrixc -emit-c` writes the optimized program as a C11 translation unit that GCC or Clang compiles like any other. Every instruction becomes a loop nest that performs the VM’s arithmetic. A product entry is summed from $+0$ over k ascending, written $i-k-j$ with literal trip counts so that the C compiler may still vectorize across j , interchange loops, or run rows on separate threads: none of these changes any entry’s order of summation. The program reads its inputs from a file, checks them against their domains as the VM does, and prints every output in hexadecimal.

For each product the facts decide the weakest level at which its sums may be reordered: bit-identical by Corollary 4.4; bound-preserving when no partial sum can overflow, since every order keeps the bound $\gamma_d|a|^T|b|$; relaxed otherwise. When the contract admits that level and the output is narrow (at most four columns), the emitter writes the product as dot products under `#pragma omp simd reduction(+:acc)`. OpenMP leaves the order in which such partial results are combined unspecified [5], so the clause is a standard, portable license to reassociate exactly that sum and no other. Compiled without OpenMP, the pragmas are ignored and every sum keeps the VM’s order. The emitted comment states the level and its reason:

```
/* t5 = A * t3  order of summation free,
   bit-identical: every term and partial sum is
   exact, so every order gives the same bits */
```

The backend is also a second implementation of the VM’s arithmetic, compiled by a different compiler. On 83 programs (every example and generated programs over all domains), under each contract, GCC’s build of the emitted C printed exactly the VM’s output for the same optimized program, with and without OpenMP SIMD; under the strict contract it printed the *unoptimized* program’s output also with every licensed reduction split across vector lanes and across four threads (0 mismatches in all). Under the bounded and algebraic contracts, licensed reductions changed the output of 27 and 33 programs.

9 Implementation

The compiler is 7,700 lines of C11, flex and bison. The fact domain and the segmented chain pass take 1,343 of them (`facts.c`, `chain.c`); input domains and their load-time checks take 555 (`domain.c`, `inputs.c`); contracts, side conditions and certificates are part of `optimize.c`, and the C backend is `emit_c.c`. The command line selects `-fp-strict` (default), `-fp-bounded` or `-fp-algebraic`; `-no-proofs` disables facts and reproduces the previous strict optimizer; `-certificate` prints per-output guarantees; `-random-inputs seed` draws inputs reproducibly; and `-emit-c` writes C. A browser workspace runs the same compiler locally, shows the three contracts and each output’s

certificate, checks every certificate against an unoptimized run, and presents the measurements of Section 10.

10 Evaluation

We ask: (RQ1) how much of the algebraic optimization each contract recovers with proofs; (RQ2) whether certificates are sound; (RQ3) how close the threshold of Theorem 4.3 is to the true boundary; (RQ4) what certificates are worth to a production compiler; (RQ5) which tests notice a missing side condition; and (RQ6) whether cheaper bracketings are more accurate. All scripts, seeds and raw observations are in the artifact (make contracts); every number below is generated from them. Experiments ran on an Intel Core Ultra 9 275HX (24 cores, AVX2) under Windows 11 with GCC 15.2.

Workloads. No benchmark suite or dataset could be downloaded for this study. RQ1, RQ2 and RQ5 use generated straight-line programs whose matrices are inputs over one value domain (bool to int32, real(1), unbounded real, or a per-matrix mixture), with six shape profiles (heterogeneous, square, narrow, balanced, elementwise, and an algebraic profile that adds identity, zero, negation and negative-scaling expressions). Two literal populations reproduce an earlier study: small integers (“dyadic”) and values spread over $10^{-12}..10^{12}$ (“broad”). RQ4 uses six kernels written as the mathematics is written, on generated data.

10.1 RQ1: Recovery

Table 3 and Figure 1 report 60,000 compilations with no failures. Without proofs, strict optimization removes almost nothing. With proofs it recovers 98.8% of the algebraic saving on bool, 83.6% on int8 and 77.8% on uint8 while keeping every output bit-identical; on int16 and int32 the 53-bit budget is exhausted by longer chains and recovery falls to 34.4% and 35.8%. The bounded contract recovers the entire saving on every domain with a magnitude bound, and 21.2% on unbounded real, where it cannot exclude overflow. On the earlier study’s dyadic literals the strict contract recovers 90.2%: those programs were exact all along, which explains why an earlier algebraic mode never changed their output. How far the proofs reach depends on the domain’s width. Recompiling 200 heterogeneous and narrow programs with every matrix over $\text{int}(-(2^b - 1), 2^b - 1)$, the strict contract proves bit-identical 99% of the chain reorderings the algebraic contract applies at $b = 3$, 89% at $b = 7$, 46% at $b = 15$, 3% at $b = 17$, and none from $b = 27$: a product of three such operands already needs more than 53 bits. The proofs are cheap: compiled one at a time, a sample of 300 of these programs took a median 8.5 ms end to end under strict against 8.5 ms without proofs (median ratio 1.00, largest 1.11), a time dominated by process start-up.

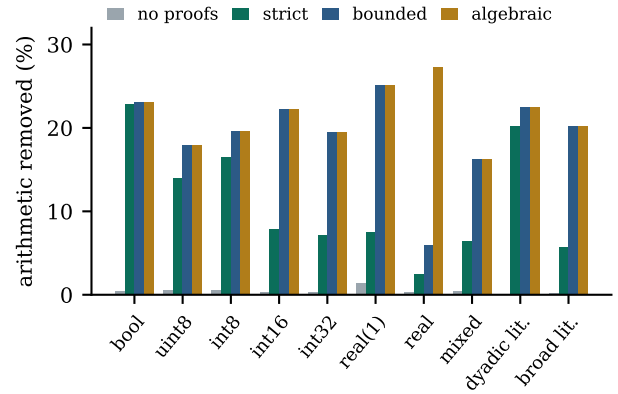


Figure 1. RQ1. Modeled arithmetic removed per value domain and contract.

10.2 RQ2: Soundness of certificates

Each program ran unoptimized and under every contract on three seeded input draws, 57,600 executions in all (Table 4). Strict output equalled unoptimized output on all 57,600 outputs. Under the bounded and algebraic contracts, 0 and 0 outputs certified bit-identical differed; the outputs that did differ were all certified bound-preserving or relaxed. This experiment is what found the inheritance defect of Section 3: before the fix, 12 outputs per contract, from four programs of the algebraic profile, were certified bit-identical and differed. Execution checks a certificate for drawn inputs only; the certificate itself is a claim over the whole domain, which Theorem 4.3 supports. Certificates are also conservative: of 3,995 outputs the bounded contract certified below bit-identical, 870 (21.8%) printed the unoptimized bits on all three draws; the certificate bounds what may change, not what will.

10.3 RQ3: Tightness of the threshold

For four chain families over $\text{int}(0, m)$ we found the largest m for which strict reorders by binary search on the compiler’s certificates (Table 5). It equals the closed form of Theorem 4.3 in every family; for the four-operand chain it is the threshold of the first segment. Allowing one final rounding raises every threshold over the exact-only condition, by 0.3–1.4%. We then looked for witnesses at 12 values of m per family, from $m^* - 3$ to $2m^*$: inputs on which the refused bracketing, written out explicitly and run unoptimized, prints different bits from the source order. Besides generic inputs (all-maximum, six near-maximum and ten seeded draws), a search emulates the VM’s arithmetic on one output entry and tries inputs at the maximum with a few entries lowered, so that a partial sum before the last one is odd and above 2^{53} and therefore rounds; every witness it finds is confirmed in the VM. No input at or below a threshold separated the bracketings (0 witnesses), and none of the 847 strict runs differed from

Table 3. RQ1. Modeled arithmetic removed under each contract (pooled over 15,000 programs, ten seeds, six profiles), and recovery: the share of the algebraic contract’s saving obtained over the unproved strict optimizer.

Domain	Modeled arithmetic removed (%)				Recovery (%)	
	no proofs	strict	bounded	algebraic	strict	bounded
bool	0.4	22.8	23.1	23.1	98.8	100.0
uint8	0.5	14.0	17.9	17.9	77.8	100.0
int8	0.5	16.5	19.6	19.6	83.6	100.0
int16	0.3	7.8	22.3	22.3	34.4	100.0
int32	0.3	7.1	19.4	19.4	35.8	100.0
real(1)	1.4	7.5	25.1	25.1	25.9	100.0
real	0.2	2.5	6.0	27.2	8.4	21.2
mixed	0.4	6.4	16.3	16.3	37.9	100.0
dyadic literals	0.1	20.2	22.4	22.4	90.2	100.0
broad literals	0.2	5.7	20.2	20.2	27.5	100.0

Table 4. RQ2. Outputs of 4,800 executed programs on three input seeds, compared bit for bit with the unoptimized program.

Contract	Outputs	Differ	Bit-identical		Bound-preserving	
			certified	differ	certified	differ
strict	57,600	0	57,600	0	0	0
bounded	57,600	9,202	45,615	0	11,985	9,202
algebraic	57,600	10,816	43,827	0	11,877	9,202

Table 5. RQ3. Largest m for which strict reorders $\text{int}(0, m)$ chains: under the exact-only condition, by Theorem 4.3 in closed form, and found by binary search on the compiler’s certificates; the smallest m with a confirmed witness; strict runs that differed from unoptimized output.

Chain	exact only	Thm. 1	compiler	witness	differ
100×2	35,578	35,697	35,697	m^*+1	0
50×4	35,578	35,818	35,818	m^*+1	0
40×8	30,419	30,676	30,676	m^*+1	0
24×3×4	50,013	50,727	50,727	m^*+2	0

unoptimized output. For the three three-operand families a witness exists at $m^* + 1$: the threshold is the true boundary. For the four-operand chain the first witness is at $m^* + 2$, and none can exist at $m^* + 1$: that maximum is even, so an odd partial sum needs a lowered entry in the first operand’s row, which brings the sum below 2^{53} . Generic inputs alone found their first witness no lower than $1.100 m^*$, which is why an earlier version of this study, which lacked the search, reported up to ten percent of slack.

10.4 RQ4: Value in a production compiler

Table 6 compiles six kernels written as one would write the mathematics: walk counts $A^3 1$ in a 1,024-vertex graph (bool adjacency), two stacked quantized layers $W_2 W_1 x$, row and

column profiles of a 2,048² 8-bit image, the normal equations $X^T X$, $X^T y$ of an integer-coded regression, and, as real-valued controls, a rank-16 update $B A x$ and three diffusion steps $P^3 v$. Times are medians of 50 runs in five processes. The source build printed the VM’s unoptimized output on every kernel.

Fast-math gains at most 1.13×: GCC reassociates scalar sums but does not reorder matrix chains. The strict contract proves the reorderings of walks and layers bit-identical and gains 278× and 4.98× with identical bits. It declines the real-valued chains, which the bounded contract reorders within the source order’s error bound for 469× and 277×. Licensed reductions add little on these kernels (0.93–1.10× over strict on one thread). The products they apply to are matrix–vector products, which use each matrix entry once, so data movement rather than the order of additions bounds their speed. Every integer kernel printed one result across all eleven builds, from SSE2 to eight threads, as Theorem 4.3 guarantees. Each real-valued kernel printed two: the source order’s, and the reordered chain’s, which the bounded and algebraic builds shared on this machine although nothing guarantees it.

The license pays where a reduction is the only parallelism. We compiled six hand-written reductions twice with GCC: strict, and with `-fassociative-math`, `-fno-signed-zeros` and `-fno-trapping-math`, which let GCC reassociate and vectorize them (Table 7). On certified data both builds printed identical bits, and the licensed build was faster by 1.23× (an `int16` dot product, memory-bound) to 6.54× (an `int8` dot product in `binary32`, compute-bound). The real-valued control changed bits for a comparable speedup. A run-time domain check over both inputs of the `int16` dot product costs 95.1% of the strict kernel; MatrixLang performs it once at load time. With OpenMP, whose reduction combines partial sums in an unspecified order, the certified dot product gave one result for every thread count from 1 to 24 and every repetition, while the real-valued one gave thirteen distinct results.

Table 6. RQ4. Six kernels compiled by `matrixc` to C and by GCC 15.2 (`-O3 -march=native -ffp-contract=off`): median time of the program as written, and speedups over it of the same C under `-ffast-math`; of the strict contract; of strict with licensed reductions (`-fopenmp-simd`); of the bounded contract with licensed reductions; and of strict with licensed reductions on eight threads. [†] marks outputs that differ from the source program. Results counts distinct outputs over eleven builds, including SSE2-only builds and the algebraic contract.

Kernel	Inputs	Source (ms)	Speedup over source					Results
			fast-math	strict	+ clauses	bounded	8 threads	
walks	bool	342	1.12	278	288	288	1,202	1
layers	int8, uint8	1.14	0.99	4.98	5.47	5.44	14.6	1
image	uint8	4.61	1.02	0.98	1.02	1.05	1.87	1
normal	int8, int16	0.0288	1.09	1.27	1.28	1.24	1.31	1
lowrank	real(1)	11.8	1.01	1.00	0.93	469 [†]	6.51	2
diffusion	real(0,1)	325	1.13	1.06	1.05	277 [†]	5.06	2

Table 7. RQ4, hand-written reductions on integer-valued binary64 (binary32 for `fdot`), sized so every partial sum is exact, compiled by GCC 15.2 `-O3 -march=native` strict and with reassociation licensed. Medians of 7 processes $\times$ 21 repetitions, one thread.

Kernel	strict (μ s)	licensed (μ s)	speedup	same bits
dot_i16	414.6	337.8	1.23 $\times$	yes
sum_i32	830.9	337.8	2.46 $\times$	yes
matvec_u8_i16	2,504.4	1,460.3	1.71 $\times$	yes
gemm_i8	4,095.1	1,259.3	3.25 $\times$	yes
fdot_i8	91.6	14.0	6.54 $\times$	yes
dot_real	412.4	341.5	1.21 $\times$	no

10.5 RQ5: Test adequacy by mutation

Ten mutants each drop one side condition (Table 8). Small-integer programs, the population an earlier differential study relied on, kill no test of mutant 1, which reorders without any proof: by Theorem 4.3 those programs are bit-identical under every bracketing, so the population is structurally blind to the defect. Broad-valued literals kill it in most programs. Side conditions on `-0` and non-finite values are rarely exercised by random programs; one witness program per condition, written from Table 2, kills mutants 1, 2, 4, 6, 7, 8 and 9. Mutant 5 is equivalent: a non-finite fact never carries “no `-0`”, so the `-0` condition already blocks every rewrite it would admit. Mutant 2, which checks only the whole chain, and mutant 10, which lets every sub-chain round once instead of only the whole chain, are each killed by a few random programs (2 broad-valued and 4 domain tests for mutant 2, 2 and 4 for mutant 10). Mutant 3, which accepts 54 bits, survives every population whose inputs are drawn at random or at the domain’s bounds, all-maximum inputs included. Only the boundary population, the witnesses of RQ3’s search placed just above each threshold, kills it (24 of 26 tests). A test suite for a numerical optimizer needs inputs constructed from its proof obligations.

10.6 RQ6: Accuracy of cheaper bracketings

Theorem 5.1 says the cheaper bracketing has the same worst-case bound; it says nothing about typical error. We preregistered a study before running it: chains of three to five real-valued operands with dimensions up to 200, values uniform in $[-1, 1]$, uniform in $[0, 1]$, or broad; ten new seeds; error $\max_{ij} |\hat{c}_{ij} - c_{ij}| / (|A_1| \cdots |A_k|)_{ij} / u$ against an exact integer reference; and a numpy emulation of the VM’s evaluation order, which matched `matrixc` bit for bit on 120 of 120 validation bracketings. The prediction was that the pooled sign test rejects equality at 0.01 in favour of the cost-optimal order with the same direction in every profile, and that an equal-cost ablation removes the effect.

The cost-optimal bracketing was more accurate in 728 of 1,222 chains with unequal error (Holm $p = 4.6 \times 10^{-11}$; median ratio 0.941, 95% CI [0.925, 0.955]), significantly on heterogeneous shapes (411 of 635, ratio 0.892) and in the same direction but not significantly on narrow shapes (317 of 587, $p = 0.058$), where cheaper bracketings save less. Pairs of distinct bracketings with equal modeled cost showed no difference (652 of 1,296, $p = 0.85$), as an operation-count account predicts. Because equal-cost pairs almost always also have equal intermediate sizes, the ablation does not separate that account from the rival that smaller intermediates cause the effect; we report the association, not the mechanism. No chain exceeded the bound of Theorem 5.1; the largest error reached 0.491 of it.

11 Related work

Fast-math semantics. GCC’s `-fassociative-math` requires `-fno-signed-zeros` and `-fno-trapping-math` [3], and LLVM attaches `reassoc`, `nsz`, `nnan` and other permissions to individual instructions [4]. Permissions are finer than a global switch but still state what may change, not what holds. Icing gives fast-math style optimizations a semantics in a verified compiler [8], and RealCake verifies such optimizations against a real-valued semantics with a proved

Table 8. RQ5. Tests that kill each mutant (number of tests per population in parentheses). A test kills a mutant when the mutant’s strict-optimized output differs from the correct unoptimized output.

Mutant	dyadic (90)	broad (90)	domain (720)	extreme (720)	fixtures (15)	witness (7)	boundary (26)
1 reorder without proof	0	51	247	96	2	2	26
2 whole chain only	0	2	4	0	0	1	0
3 54-bit threshold	0	0	0	0	0	0	24
4 AI: ignore -0	3	0	9	22	0	1	0
5 AI: ignore non-finite	0	0	0	0	0	0	0
6 A + 0: ignore -0	1	1	6	21	1	2	0
7 A0: ignore non-finite	0	0	0	3	0	2	0
8 0A: ignore sign	4	7	45	47	1	1	0
9 negation yields no -0	4	1	15	20	0	3	0
10 one rounding in every sub-chain	0	2	4	0	0	0	0

Table 9. RQ6. Paired comparison of the cost-optimal and source bracketings on real-valued chains, and the equal-cost ablation. Sign tests are two-sided and exact; Holm-adjusted over the three confirmatory tests; median ratio of errors (cost-optimal / source) with seed-cluster bootstrap 95% intervals.

Population	chains	unequal	cheaper better	p (Holm)	median ratio [95% CI]
heterogeneous	648	635	411	3.1×10^{-13}	0.892 [0.863, 0.913]
narrow	606	587	317	0.058	0.979 [0.954, 1.000]
pooled	1,254	1,222	728	4.6×10^{-11}	0.941 [0.925, 0.955]
<i>Equal-cost ablation: first = fewer intermediate entries, or enumeration order</i>					
heterogeneous	64	59	27	0.6	
narrow	363	352	199	0.016	
square	900	885	426	0.28	
pooled	1,327	1,296	652	0.85	

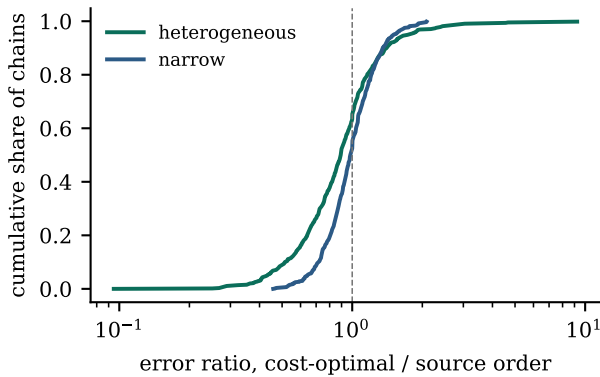


Figure 2. RQ6. Distribution of the error ratio (cost-optimal / source) over chains whose cheaper bracketing differs from source order.

error bound [9]; there the program opts in and the bound is the guarantee. Our levels are derived rather than granted, so a strict program gets reassociation exactly where it is provably bit-identical. Alive and Alive-FP verify peephole optimizations, including floating-point preconditions on signed zeros

and non-finite values [10, 11]; Alive-Infer infers such preconditions [12]; Alive2 validates LLVM translations [13]. Our side conditions (Table 2) are preconditions of this kind; our contribution is their organisation into guarantee levels, their discharge from domain facts, and per-output certificates. Boldo et al. verify that CompCert preserves floating-point semantics exactly [14]; Monniaux catalogues why compiled floating-point code is hard to reason about [15].

Value facts in production compilers. GCC’s ranger tracks floating-point ranges together with whether a value may be NaN and the sign of its zeros [16], and LLVM’s KnownFPClass analysis, since LLVM 17, tracks whether a value may be NaN, infinite or -0 [17]. Both can discharge side conditions like those of Table 2 for single instructions. LLVM also folds floating-point arithmetic on integer-to-float conversions into integer arithmetic when both are exact [18], a one-operation instance of our exactness argument. None of these tracks the grid of a value, proves a reassociation bit-identical, or certifies outputs.

Accuracy analysis and repair. Rosa and Daisy compile real-valued specifications to finite precision under error bounds [19, 20]; FPTaylor bounds round-off of expressions [21]; Herbie rewrites expressions for accuracy [22].

CIGEN searches for input ranges on which optimization levels make results differ [23]; we prove, for declared ranges, which rewrites cannot.

Exactness and reproducibility. The Ozaki scheme splits floating-point matrices into slices whose products are computed without error [24] and emulates binary64 GEMM on integer units [25, 26]; FFLAS-FFPACK bounds intermediates below 2^{53} to do exact integer linear algebra on floating-point units [27]. These libraries rest on the arithmetic fact that Theorem 4.3 turns into a compiler legality criterion over chains of any bracketing, with signed zeros, sub-chains and one final rounding. Error-free transformations make sums exact or accurate [28]; reproducible summation makes results independent of order at a cost [29, 30]; Intel’s oneMKL reproduces results run to run for a fixed thread count and code path [31]; FPRev shows that libraries’ accumulation orders differ [32]. Where our certificates apply, reproducibility across thread counts and vector widths costs nothing. Bitwidth analysis computes value ranges to size hardware [33]. The bracketing invariance of Theorem 5.1 follows from the standard analysis of inner products [6]; we claim its use as a contract, not the inequality.

Matrix chains and testing. Optimal chain ordering is classical [7]; Linnea generates linear-algebra code with property-aware cost models [34]. Neither certifies numerical guarantees per output. Csmith-style differential testing [35] and mutation analysis [36] are established; RQ5 shows a structural reason why value-blind differential testing misses reassociation defects.

12 Limitations

MatrixLang is straight-line and statically shaped; loops would need the facts to reach a fixed point, which the lattice permits but we have not implemented. The bounded contract promises the standard-model bound; with gradual underflow both orders need an absolute error term, which we do not show to be invariant. Grids of real-valued data are 2^{-1074} , so bit-identity proofs apply to integer and dyadic data, not to general reals. Domains are trusted declarations checked at load time; a domain that is wider than the data loses opportunities, never soundness. The threshold is the true boundary for the chain families of RQ3, not in general: magnitude bounds cannot see, for instance, that every input is even. Workloads are generated or written by us, not drawn from applications, because no dataset or benchmark suite was available offline, and RQ4 measures one compiler on one machine. The equal-cost ablation in RQ6 cannot discriminate between two mechanisms. The C backend checks the VM’s arithmetic independently but shares its front end and optimizer.

13 Conclusion

A floating-point rewrite keeps one of three guarantees, and which one is often provable from the values a program will see. Declared input domains, a grid-magnitude analysis and two theorems let a compiler reorder chains under strict semantics where every order returns the correctly rounded product, under a bounded contract where the error bound is unchanged, and report the guarantee of every output. The same facts tell a production compiler which sums it may reorder. Checking certificates against execution found a real defect, and inputs built from the proof obligations killed a mutant that random inputs missed. Under GCC the licensed reorderings made one integer kernel 278 times faster with identical bits and two real-valued kernels 277 and 469 times faster within the source order’s error bound, where `-ffast-math` gained at most $1.13\times$.

Data availability

The compiler, experiment scripts, raw observations and workspace are in the artifact repository. `make contracts` repeats every measurement and `make contracts-analyze` regenerates every number, table and figure.

References

- [1] David Goldberg. 1991. What Every Computer Scientist Should Know About Floating-Point Arithmetic. *ACM Comput. Surv.* 23, 1 (1991), 5–48. <https://doi.org/10.1145/103162.103163>
- [2] IEEE. 2019. *IEEE Standard for Floating-Point Arithmetic*. IEEE Std 754-2019.
- [3] Free Software Foundation. GCC Manual: Options That Control Optimization (`-fassociative-math`, `-fno-signed-zeros`, `-fno-trapping-math`). <https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html>
- [4] LLVM Project. LLVM Language Reference Manual: Fast-Math Flags. <https://llvm.org/docs/LangRef.html#fast-math-flags>
- [5] OpenMP Architecture Review Board. 2021. *OpenMP Application Programming Interface, Version 5.2*, Section 5.5.6, Reduction Scoping Clauses. <https://www.openmp.org/spec-html/5.2/openmp50.html>
- [6] Nicholas J. Higham. 2002. *Accuracy and Stability of Numerical Algorithms* (2nd ed.). SIAM.
- [7] T. C. Hu and M. T. Shing. 1982. Computation of Matrix Chain Products. Part I. *SIAM J. Comput.* 11, 2 (1982), 362–373.
- [8] Heiko Becker, Eva Darulova, Magnus O. Myreen, and Zachary Tatlock. 2019. Icing: Supporting Fast-Math Style Optimizations in a Verified Compiler. In *CAV 2019*, 155–173. https://doi.org/10.1007/978-3-030-25543-5_10
- [9] Heiko Becker, Robert Rabe, Eva Darulova, Magnus O. Myreen, Zachary Tatlock, Ramana Kumar, Yong Kiam Tan, and Anthony Fox. 2022. Verified Compilation and Optimization of Floating-Point Programs in CakeML. In *ECOOP 2022*, LIPIcs 222, 1:1–1:28. <https://doi.org/10.4230/LIPIcs.ECOOP.2022.1>
- [10] Nuno P. Lopes, David Menendez, Santosh Nagarakatte, and John Regehr. 2015. Provably Correct Peephole Optimizations with Alive. In *PLDI*.
- [11] David Menendez, Santosh Nagarakatte, and Aarti Gupta. 2016. Alive-FP: Automated Verification of Floating Point Based Peephole Optimizations in LLVM. In *SAS*.
- [12] David Menendez and Santosh Nagarakatte. 2017. Alive-Infer: Data-Driven Precondition Inference for Peephole Optimizations in LLVM.

- In *PLDI*.
- [13] Nuno P. Lopes, Juneyoung Lee, Chung-Kil Hur, Zhengyang Liu, and John Regehr. 2021. Alive2: Bounded Translation Validation for LLVM. In *PLDI*, 65–79. <https://doi.org/10.1145/3453483.3454030>
- [14] Sylvie Boldo, Jacques-Henri Jourdan, Xavier Leroy, and Guillaume Melquiond. 2015. Verified Compilation of Floating-Point Computations. *J. Autom. Reason.* 54, 2 (2015), 135–163.
- [15] David Monniaux. 2008. The Pitfalls of Verifying Floating-Point Computations. *ACM Trans. Program. Lang. Syst.* 30, 3 (2008).
- [16] Aldy Hernandez. 2023. How We Expanded GCC Value Range Propagation for Floats. Red Hat Developer. <https://developers.redhat.com/articles/2023/01/18/how-we-expanded-gcc-value-range-propagation-floats>
- [17] LLVM Project. 2023. computeKnownFPClass and KnownFPClass, `llvm/include/llvm/Analysis/ValueTracking.h`, release 17. <https://github.com/llvm/llvm-project/blob/release/17.x/llvm/include/llvm/Analysis/ValueTracking.h>
- [18] LLVM Project. 2024. InstCombine `foldFBinOpOfIntCasts`, `llvm/lib/Transforms/InstCombine`, release 18. <https://github.com/llvm/llvm-project/tree/release/18.x/llvm/lib/Transforms/InstCombine>
- [19] Eva Darulova and Viktor Kuncak. 2014. Sound Compilation of Reals. In *POPL*.
- [20] Eva Darulova, Anastasiia Izycheva, Fariha Nasir, Fabian Ritter, Heiko Becker, and Robert Bastian. 2018. Daisy – Framework for Analysis and Optimization of Numerical Programs. In *TACAS*.
- [21] Alexey Solovyev, Charles Jacobsen, Zvonimir Rakamarić, and Ganesh Gopalakrishnan. 2015. Rigorous Estimation of Floating-Point Round-off Errors with Symbolic Taylor Expansions. In *FM*.
- [22] Pavel Panchekha, Alex Sanchez-Stern, James R. Wilcox, and Zachary Tatlock. 2015. Automatically Improving Accuracy for Floating Point Expressions. In *PLDI*.
- [23] Dolores Miao, Ignacio Laguna, and Cindy Rubio-González. 2024. Input Range Generation for Compiler-Induced Numerical Inconsistencies. In *ICS '24*, 201–212. <https://doi.org/10.1145/3650200.3656618>
- [24] Katsuhisa Ozaki, Takeshi Ogita, Shin'ichi Oishi, and Siegfried M. Rump. 2012. Error-Free Transformations of Matrix Multiplication by Using Fast Routines of Matrix Multiplication and Its Applications. *Numer. Algorithms* 59, 1 (2012), 95–118. <https://doi.org/10.1007/s11075-011-9478-1>
- [25] Hiroyuki Ootomo, Katsuhisa Ozaki, and Rio Yokota. 2024. DGEMM on Integer Matrix Multiplication Unit. *Int. J. High Perform. Comput. Appl.* 38, 4 (2024), 297–313. <https://doi.org/10.1177/10943420241239588>
- [26] Katsuhisa Ozaki, Yuki Uchino, and Toshiyuki Imamura. 2026. Ozaki Scheme II: A GEMM-Oriented Emulation of Floating-Point Matrix Multiplication Using an Integer Modular Technique. *Int. J. High Perform. Comput. Appl.* <https://doi.org/10.1177/10943420261467787>
- [27] Jean-Guillaume Dumas, Pascal Giorgi, and Clément Pernet. 2008. Dense Linear Algebra over Word-Size Prime Fields: the FFLAS and FFPACK Packages. *ACM Trans. Math. Softw.* 35, 3 (2008).
- [28] Takeshi Ogita, Siegfried M. Rump, and Shin'ichi Oishi. 2005. Accurate Sum and Dot Product. *SIAM J. Sci. Comput.* 26, 6 (2005).
- [29] James Demmel and Hong Diep Nguyen. 2013. Fast Reproducible Floating-Point Summation. In *ARITH*, 163–172.
- [30] Willow Ahrens, James Demmel, and Hong Diep Nguyen. 2020. Algorithms for Efficient Reproducible Floating Point Summation. *ACM Trans. Math. Softw.* 46, 3 (2020). <https://doi.org/10.1145/3389360>
- [31] Intel. 2025. oneMKL Developer Guide: Obtaining Numerically Reproducible Results. <https://www.intel.com/content/www/us/en/docs/onemkl/developer-guide-linux/2025-2/obtaining-numerically-reproducible-results.html>
- [32] Peichen Xie, Yanjie Gao, Yang Wang, and Jilong Xue. 2025. Revealing Floating-Point Accumulation Orders in Software/Hardware Implementations. In *USENIX ATC '25*, 1425–1440.
- [33] Mark Stephenson, Jonathan Babb, and Saman Amarasinghe. 2000. Bitwidth Analysis with Application to Silicon Compilation. In *PLDI*.
- [34] Henrik Barthels, Christos Psarras, and Paolo Bientinesi. 2021. Linnea: Automatic Generation of Efficient Linear Algebra Programs. *ACM Trans. Math. Softw.* 47, 3 (2021). <https://doi.org/10.1145/3446632>
- [35] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and Understanding Bugs in C Compilers. In *PLDI*. <https://doi.org/10.1145/1993316.1993532>
- [36] Yue Jia and Mark Harman. 2011. An Analysis and Survey of the Development of Mutation Testing. *IEEE Trans. Softw. Eng.* 37, 5 (2011).